

Database Systems-SQL

Nitin Awathare

Department of Computer Science and Engineering,
IIT Jodhpur

1



Introduction to SQL

- CREATE TABLE
 - Create a new table, e.g., students, courses, enrolled
- SELECT-FROM-WHERE
 - List all CS courses
- INSERT
 - Add a new student, course, or enroll a student in a course

Create Table

- CREATE TABLE Enrolled

(studid CHAR(20),

cid CHAR(20),

grade CHAR(20),

PRIMARY KEY (studid, cid),

FOREIGN KEY (studid) references Students)

Select From Table

- “Find all students who are under 18”

```
SELECT *
```

```
FROM Students S
```

```
WHERE S.age < 18
```

Query Across Multiple Tables

- “Print the student name and course ID where the student received an ‘A’ in the course”

```
SELECT S.name, E.cid
```

```
FROM   Students S, Enrolled E
```

```
WHERE  S.sid = E.studid AND E.grade = ‘A’
```

Other SQL Features

- MIN, MAX, AVG
 - Find highest grade in fall database course
- COUNT, DISTINCT
 - How many students enrolled in CS courses in the fall?
- ORDER BY, GROUP BY
 - Rank students by their grade in fall database course

Aggregate Function

- Find the average salary of instructors in the Computer Science department
 - **select avg** (*salary*)
from *instructor*
where *dept_name*= 'Comp. Sci.';
- Find the total number of instructors who teach a course in the Spring 2018 semester
 - **select count** (**distinct** *ID*)
from *teaches*
where *semester* = 'Spring' **and** *year* = 2018;
- Find the number of tuples in the *course* relation
 - **select count** (*)
from *course*;

Aggregate Function

- Find the average salary of instructors in each department
 - ***select dept_name, avg (salary) as avg_salary***
from instructor
group by dept_name;

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Aggregate Function

- Attributes in **select** clause outside of aggregate functions must appear in **group by** list
 - */* erroneous query */*
select *dept_name, ID, avg (salary)*
from *instructor*
group by *dept_name*;
- Find the names and average salaries of all departments whose average salary is greater than 42000
 - **select** *dept_name, avg (salary) as avg_salary*
from *instructor*
group by *dept_name*
having **avg** (*salary*) > 42000;

Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A **subquery** is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```

as follows:

- **From clause:** r_i can be replaced by any valid subquery
- **Where clause:** P can be replaced with an expression of the form:

B <operation> (subquery)

B is an attribute and <operation> to be defined later.

- **Select clause:**

A_i can be replaced by a subquery that generates a single value.

Set Operations

- Find courses that ran in Fall 2017 or in Spring 2018

(select *course_id* from *section* where *sem* = 'Fall' and *year* = 2017)

union

(select *course_id* from *section* where *sem* = 'Spring' and *year* = 2018)

- Find courses that ran in Fall 2017 and in Spring 2018

(select *course_id* from *section* where *sem* = 'Fall' and *year* = 2017)

intersect

(select *course_id* from *section* where *sem* = 'Spring' and *year* = 2018)

- Find courses that ran in Fall 2017 but not in Spring 2018

(select *course_id* from *section* where *sem* = 'Fall' and *year* = 2017)

except

(select *course_id* from *section* where *sem* = 'Spring' and *year* = 2018)

Views

- Virtual table defined on base tables defined by a query
 - Single or multiple tables
- Security – “hide” certain attributes from users
 - Show students in each course but hide their grades
- Ease of use – expression that is more intuitively obvious to user
- Views can be materialized to improve query performance

Views

- Suppose we often need names of students who got a 'B' in some course:

```
CREATE VIEW B_Students(name, sid, course)
AS SELECT S.sname, S.sid, E.cid
FROM Students S, Enrolled E
WHERE S.sid=E.studid and E.grade = 'B'
```

name	sid	course
Jones	53666	History105
Guldu	53832	Reggae203

Indexes

- Idea: speed up access to desired data
- “Find all students with $\text{gpa} > 3.3$ ”
- May need to scan entire table
- Index consists of a set of *entries* pointing to locations of each *search key*

Type of Indexes

■ Clustered vs. Unclustered

- Clustered- ordering of data records same as ordering of data entries in the index
- Unclustered- data records in different order from index

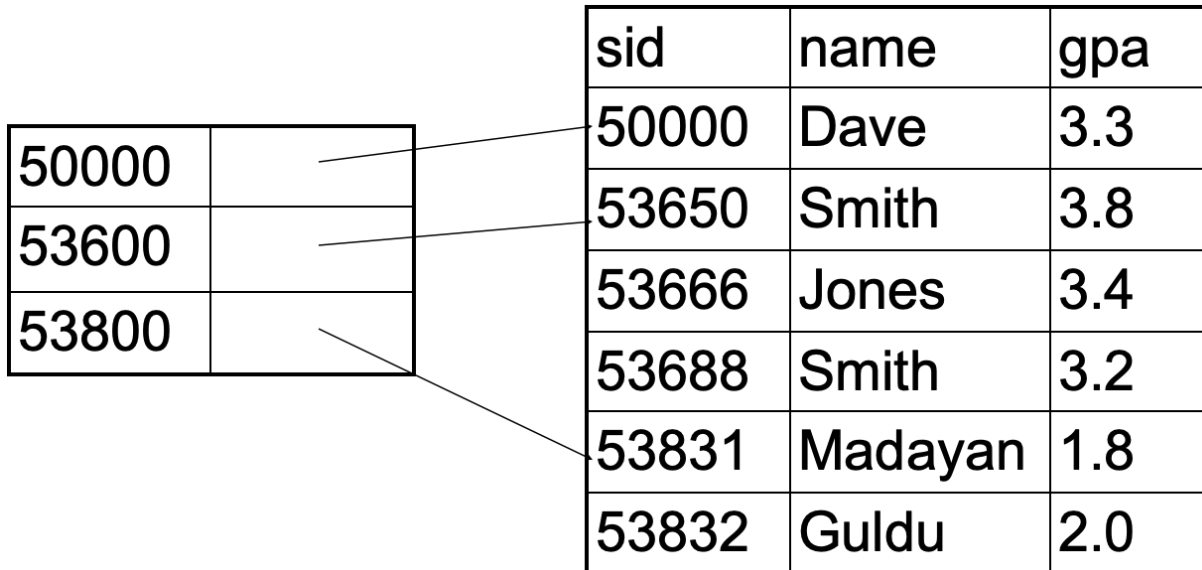
■ Primary vs. Secondary

- Primary – index on fields that include primary key
- Secondary – other indexes

Clustered Index

- Sorted by sid

50000		50000	Dave	3.3
53600		53650	Smith	3.8
53800		53666	Jones	3.4
		53688	Smith	3.2
		53831	Madayan	1.8
		53832	Guldu	2.0



Unclustered Index

- Sorted by sid
- Index on gpa

1.8	
2.0	
3.2	
3.3	
3.4	
3.8	

sid	name	gpa
50000	Dave	3.3
53650	Smith	3.8
53666	Jones	3.4
53688	Smith	3.2
53831	Madayan	1.8
53832	Guldu	2.0



THANK YOU